

Relazione Homework 3 Sistemi Distribuiti e Big Data

OpenSky Flight Monitor

Giovanni Maria Contarino 1000007029

Alessia Provvidenza Tomarchio 1000005160

Sommario

Introduzione	4
Architettura del Sistema	4
Diagramma Architetture	5
Descrizione dei Componenti Architetture	5
Entry Point e Routing (Ingress Layer)	5
Microservizi principali	5
Comunicazioni e Gestione dei Dati	6
Monitoraggio.....	6
Pattern di Comunicazione.....	7
Comunicazione Sincrona:	7
Comunicazione Asincrona (Event-Driven):	7
Diagramma delle interazioni e continuità con le versioni precedenti	7
Descrizione dei Microservizi e Scelte Progettuali	8
User Manager.....	8
Data Collector.....	9
Alert System.....	9
Notifier e Prometheus.....	10
Shared Library	10
Continuità Funzionale e Riferimenti alla Documentazione Pregressa	10
Deploy e Orchestrazione su Kubernetes.....	11
Configurazione della Topologia del Cluster	11
Strategia di Networking: Ingress Controller	12
Gestione Dichiarativa dei Workload (Deployments)	12
Focus: Kafka in modalità KRaft	12
Configurazione Esterna	12
ConfigMaps	13
Secrets.....	13
Persistenza dello Stato	13
Sicurezza di Rete (Network Policies)	13
Automazione del Deployment	14
Monitoraggio e White-Box Monitoring.....	14
Metriche di Business	14
Monitoraggio Data Collector.....	14
Monitoraggio Notifier.....	17
Monitoraggio User Manager	17

Metriche di Sistema (Standard Metrics).....	20
Metriche di Salute dell'Infrastruttura	23
Conclusioni e Sviluppi Futuri	24
Analisi dei Risultati Raggiunti	25
Orchestrazione Dichiarativa e Self-Healing	25
Evoluzione dell'Event-Driven Design	25
Osservabilità Completa (White-Box Monitoring)	25
Limitazioni Attuali e Sviluppi Futuri.....	25
Transizione a Redis	Errore. Il segnalibro non è definito.
Alerting Proattivo	25
Appendice A: Guida all'Installazione e Deploy	26
Prerequisiti.....	26
Inizializzazione	26
Gestione Operativa (Stop & Resume).....	27
Arresto del Cluster	27
Ripristino del Cluster	27
Guida Pratica alle Query Prometheus	27
Metriche di Business	27
Metriche di Sistema e Risorse.....	28
Metriche di Salute Infrastrutturale	28
Accesso ai Database	29
Creazione del Tunnel (Port Forwarding)	29
Configurazione Data Source su IntelliJ IDEA.....	30
Configurazione PostgreSQL	30
Configurazione MongoDB.....	30

Introduzione

Il progetto **OpenSky Flight Monitor** consiste nello sviluppo di un sistema distribuito a microservizi per il monitoraggio in tempo reale del traffico aereo, l'analisi dei dati di volo e la gestione proattiva delle notifiche agli utenti.

L'obiettivo principale del sistema è permettere agli utenti registrati di definire interessi specifici su determinati aeroporti (es. Hartsfield, Heathrow, JFK) e ricevere allarmi immediati qualora il numero di voli (in arrivo o partenza) superi o scenda al di sotto di determinate soglie critiche.

Il progetto si è evoluto attraverso diverse iterazioni incrementali che hanno portato alla definizione di un'architettura progettata per essere scalabile, resiliente ai guasti e facilmente osservabile.

Nello specifico, l'attuale versione del sistema (v3.0) si caratterizza per:

- **Architettura a Microservizi:** Decomposizione funzionale in servizi autonomi (*User Manager*, *Data Collector*, *Alert System*, *Notifier*) sviluppati in Python.
- **Orchestrazione tramite Kubernetes:** L'intera infrastruttura è deployata su un cluster Kubernetes (implementato localmente tramite *Kind*), sfruttando primitive avanzate per la gestione del ciclo di vita dei container, il networking e la gestione delle configurazioni.
- **Message Broker:** Disaccoppiamento tra la fase di raccolta dati e quella di analisi tramite l'uso di **Apache Kafka**, garantendo un'elaborazione asincrona e non bloccante.
- **Persistenza e Caching Distribuito:** Utilizzo di tecnologie di storage diversificate in base al caso d'uso: **PostgreSQL** per i dati relazionali strutturati, **MongoDB** per serie storiche ad alto volume e **Redis** come store in-memory per la gestione dello stato transitorio e dell'idempotenza distribuita.
- **White-Box Monitoring:** Integrazione di **Prometheus** per la raccolta di metriche applicative in tempo reale (numero di utenti, latenza API, errori, throughput di messaggi), permettendo una visione granulare dello stato di salute del cluster.

Il sistema interagisce con il mondo esterno attraverso l'API pubblica di *OpenSky Network* per il reperimento dei dati di volo e utilizza protocolli standard (SMTP) per l'invio delle notifiche finali agli utenti.

Architettura del Sistema

L'architettura di *OpenSky Flight Monitor* (v3.0) è stata riprogettata seguendo il paradigma **Microservices Architecture (MSA)** orchestrata su container. Questa scelta progettuale risponde all'esigenza di creare un sistema modulare, in cui ogni componente possiede una responsabilità unica, è sviluppato e rilasciato indipendentemente e comunica con gli altri attraverso interfacce ben definite.

L'infrastruttura è ospitata interamente su un cluster **Kubernetes** (emulato tramite *Kind*), che astrae le risorse hardware sottostanti e gestisce il ciclo di vita delle applicazioni.

Diagramma Architettuale

Di seguito viene riportato lo schema logico del sistema, che evidenzia la distinzione tra il "Mondo Esterno" (Client e API di terze parti) e il "Cluster Kubernetes" interno.

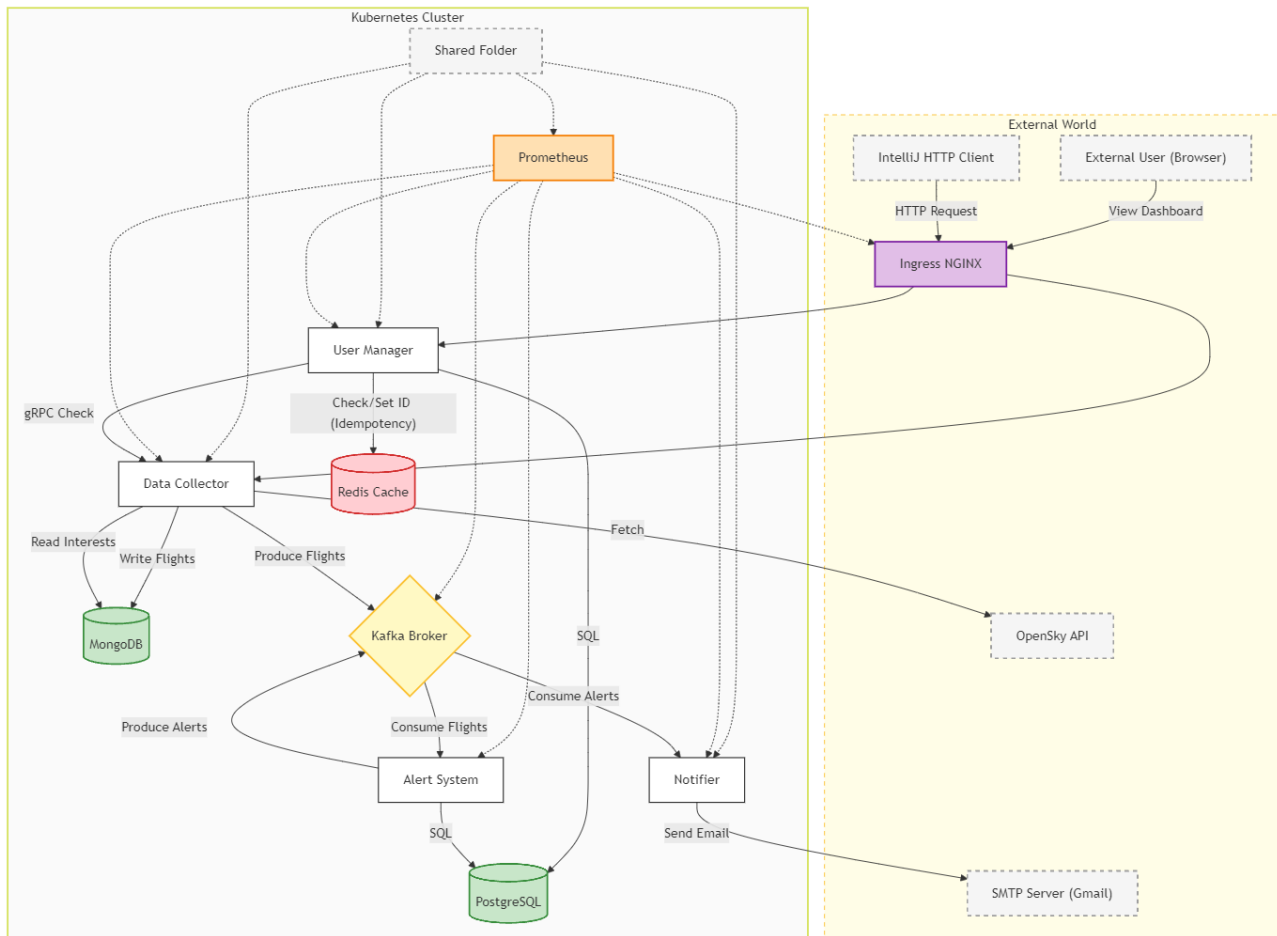


Figura 2.1 Diagramma Architettuale del nostro sistema.

Descrizione dei Componenti Architeturali

Il sistema può essere scomposto in quattro macro-livelli logici:

Entry Point e Routing (Ingress Layer)

Il punto di ingresso unico al cluster è gestito dall'**NGINX Ingress Controller**, che agisce come un Reverse Proxy. Esso:

- Termina le connessioni HTTP provenienti dai client esterni (Browser, IDE HTTP Client).
- Effettua il routing delle richieste verso i corretti servizi di backend basandosi sui path dell'URL (es. /users verso *User Manager*, /api verso *Data Collector*).
- Garantisce l'isolamento dei servizi interni, che non espongono alcuna porta direttamente su Internet.

Microservizi principali

La logica di business è distribuita su quattro microservizi Python containerizzati:

- **User Manager:** Responsabile della gestione degli utenti e dei loro dati sensibili. Espone API REST per il frontend e servizi gRPC per la comunicazione interna, con il Data Collector, ad alta efficienza.
- **Data Collector:** Si occupa del fetching periodico dei dati di volo dalle API di *OpenSky Network* e della loro iniezione nel sistema.
- **Alert System:** Analizza i flussi di dati in tempo reale per identificare violazioni delle soglie di interesse definite dagli utenti.
- **Notifier:** Il gestore delle comunicazioni in uscita. Riceve i trigger di allarme e gestisce l'invio fisico delle e-mail tramite protocollo SMTP (Gmail).

Comunicazioni e Gestione dei Dati

Per garantire scalabilità e disaccoppiamento, il livello dati è stato progettato utilizzando un approccio ibrido:

- **Persistenza:**
 1. **PostgreSQL:** Utilizzato per dati relazionali ad alta criticità, nel nostro caso per gestire i dati sensibili degli utenti registrati nel nostro sistema.
 2. **MongoDB:** Utilizzato per lo storage di dati semi-strutturati ad alto volume, come lo storico dei voli e gli interessi degli utenti.
 3. **Redis:** Introdotto come **Key-Value Store** in-memory per disaccoppiare lo stato dell'applicazione dai singoli container. Memorizza i RequestID e i lock distribuiti, garantendo che il pattern **At-Most-Once** funzioni correttamente anche quando lo *User Manager* viene scalato su più repliche (che devono condividere la "memoria" delle richieste già processate). Sebbene agisca come cache in-memory, è stato configurato come StatefulSet con persistenza su disco (appendonly yes). Questo garantisce che, in caso di riavvio del pod, lo stato delle richieste processate (RequestID) venga recuperato, preservando l'idempotenza del sistema anche a fronte di guasti temporanei.

La scelta di utilizzare MongoDB per i dati sui voli, nasce dal fatto che i dati provenienti dalle API di OpenSky Network sono nativamente in formato JSON ed hanno una struttura gerarchica. Per tale motivo, MongoDB permette di memorizzare questi dati, senza particolari manipolazioni. PostgreSQL, d'altro canto, è stato utilizzato per assicurare l'integrità dei dati utenti e gestire in sicurezza informazioni sensibili come hash delle password e dei dati bancari.

- **Message Broker (Apache Kafka):** È il cuore pulsante dell'architettura *Event-Driven*. Sostituisce le chiamate sincrone tra Data Collector e Alert System, introducendo un buffer persistente che garantisce che nessun dato di volo venga perso anche in caso di temporanea indisponibilità dei consumatori.

Monitoraggio

Un livello trasversale di monitoraggio è implementato tramite **Prometheus**. Il server Prometheus agisce in modalità *pull*, eseguendo lo scraping periodico degli endpoint `/metrics` esposti dai

microservizi User Manager, Data Collector e Notifier. Questo permette di raccogliere dati sul funzionamento di aspetti specifici del sistema, nonché informazioni relative alle risorse usate da esso stesso.

Pattern di Comunicazione

L'architettura implementa due pattern di comunicazione distinti, scelti in base alla natura dell'interazione richiesta:

Comunicazione Sincrona:

1. **Client Esterno > Ingress > Servizi:** Utilizza HTTP/1.1 REST. È bloccante e richiede una risposta immediata per l'utente.
2. **User Manager <--> Data Collector:** Utilizza gRPC. Viene impiegato per operazioni interne critiche, come la verifica dei dati degli utenti presenti all'interno del sistema.

Comunicazione Asincrona (Event-Driven):

1. **Data Collector > Kafka > Alert System > Kafka > Notifier:** Questa pipeline realizza un pattern *Producer-Consumer*. L'iter completo è il seguente:
 - **Fase 1:** Il *Data Collector* scarica i dati da OpenSky. Una volta formattati, agisce come **Producer** e pubblica il payload sul topic Kafka **flights**.
 - **Fase 2:** Kafka riceve il messaggio e lo memorizza in modo durevole. Se l'*Alert System* fosse temporaneamente offline o sovraccarico, i dati non andrebbero persi ma rimarrebbero in coda. L'*Alert System* agisce come **Consumer**. Legge i messaggi dal topic **flights** secondo il proprio ritmo di elaborazione. Confronta i dati di volo con gli interessi salvati su MongoDB.
 - **Fase 3:** Se viene rilevata una violazione (es. numero voli > soglia), l'*Alert System* diventa a sua volta **Producer** e pubblica un messaggio di allarme sul topic **to-notifier**.
 - **Fase 4:** Il *Notifier*, in ascolto sul topic **to-notifier**, preleva l'evento e contatta il server SMTP esterno per l'invio dell'email all'utente finale.

Diagramma delle interazioni e continuità con le versioni precedenti

Poiché il progetto rappresenta la terza fase evolutiva del sistema OpenSky Flight Monitor, la struttura logica fondamentale dei microservizi e le interazioni applicative standard (es. sequenza di registrazione utente, logica di estrazione voli) rimangono coerenti con quanto definito nelle versioni 1 e 2. Di conseguenza, il Diagramma delle interazioni presentato in questo capitolo si concentra sulle nuove interazioni infrastrutturali introdotte dall'orchestrazione Kubernetes. Per una visione completa dei Diagrammi delle Sequenze, si rimanda alla documentazione tecnica allegata alle relazioni degli Homework 1 e 2.

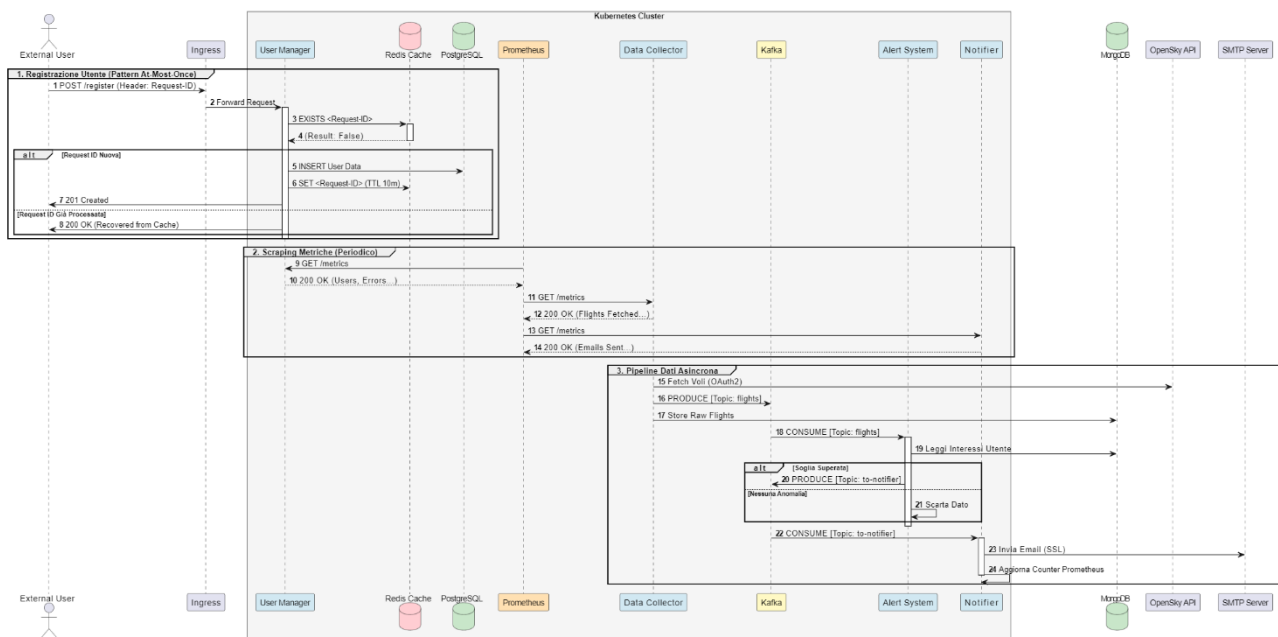


Figura 2.2 Diagramma delle interazioni tra i componenti dell'applicazione e tra l'applicazione ed il mondo esterno.

Descrizione dei Microservizi e Scelte Progettuali

Il sistema *OpenSky Flight Monitor* è costituito da **cinque microservizi** containerizzati. Quattro di questi (*User Manager*, *Data Collector*, *Alert System*, *Notifier*) implementano la logica di business e sono stati sviluppati in Python, mentre il quinto, **Prometheus**, è un componente infrastrutturale dedicato al monitoraggio *White-box*, deployato come servizio all'interno del cluster Kubernetes.

Di seguito viene fornita una descrizione tecnica dettagliata di ciascun componente, analizzando le scelte implementative e l'evoluzione subita nel corso delle diverse iterazioni del progetto.

User Manager

Lo **User Manager** è il servizio responsabile della gestione degli utenti. Oltre alle classiche operazioni CRUD, la sfida principale affrontata in questo componente è stata garantire la semantica **At-Most-Once** in un ambiente distribuito soggetto a potenziali duplicazioni di rete.

Implementazione della Cache Distribuita (Redis): Per garantire la semantica *At-Most-Once* in un ambiente distribuito, abbiamo abbandonato la cache in-memory (che vincolerebbe lo stato al singolo Pod) in favore di **Redis**. Ogni richiesta di scrittura (POST) include un header `RequestID`. Il servizio interroga Redis:

- **Hit:** Se l'ID esiste, Redis restituisce l'esito salvato precedentemente.
- **Miss:** Se l'ID è nuovo, lo User Manager processa la richiesta e salva il risultato su Redis con un TTL (Time-To-Live) di 10 minuti.

L'uso di Redis garantisce che, anche scalando lo User Manager su più repliche, l'idempotenza sia preservata globalmente. Inoltre, è stata implementata una logica di **Retry con Backoff** all'avvio: il microservizio attende attivamente che Redis sia pronto prima di accettare traffico, prevenendo race condition durante il bootstrap del cluster.

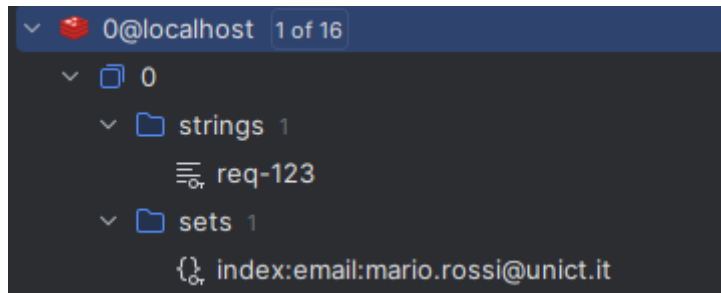


Figura 2.3 Salvataggio del RequestID associato alla email dell'utente.

Gestione della Coerenza in fase di Cancellazione: Un aspetto critico gestito riguarda la cancellazione dell'utente. Se un utente viene eliminato (DELETE /users) poco dopo la registrazione, il suo RequestID potrebbe essere ancora vivo nella cache. Se l'utente provasse a registrarsi nuovamente con lo stesso ID prima della scadenza del TTL, il sistema gli "risponderebbe" che è ancora registrato (leggendo la vecchia cache), creando un'inconsistenza. Per risolvere questo problema, la logica di cancellazione include un passaggio esplicito di **invalidazione della cache**: al momento dell'eliminazione dal DB, viene forzata la rimozione della chiave corrispondente all'RequestID associato all'email, garantendo che lo stato della cache sia sempre allineato con lo stato persistente.

Data Collector

Il **Data Collector** ha il compito di interrogare le API di terze parti (*OpenSky Network*). Poiché queste API sono soggette a rate-limiting e possibili downtime, abbiamo implementato il pattern **Circuit Breaker**, prendendo ispirazione diretta dagli esempi analizzati durante il corso.

Il componente opera secondo una macchina a stati finiti:

1. **Stato CLOSED:** È lo stato normale. Le richieste fluiscono verso OpenSky. Se si verificano **3 errori consecutivi** (es. timeout o HTTP 429), il contatore degli errori supera la soglia e il circuito "scatta".
2. **Stato OPEN:** Il circuito è aperto. Ogni tentativo di chiamata viene bloccato immediatamente (Fail Fast) lanciando un'eccezione interna, senza nemmeno tentare la connessione verso l'esterno. Questo "periodo di cooldown" (configurato per prevenire il sovraccarico del servizio remoto) dura per un intervallo di tempo di 60 secondi.
3. **Stato HALF-OPEN:** Trascorso il periodo di cooldown, il sistema entra in uno stato "probatorio". Lascia passare **una singola richiesta** di test:
 - Se ha successo, il circuito si chiude (**CLOSED**) e il contatore si azzerà.
 - Se fallisce, il circuito torna immediatamente **OPEN** per un nuovo ciclo di attesa.

Alert System

L'Alert System agisce in duplice veste:

- **Consumer:** Sottoscrive il topic flights per ricevere i dati grezzi dal Data Collector.
- **Producer:** Una volta elaborata la logica di business (confronto voli/soglie), se rileva una condizione di allarme, agisce da *Producer* pubblicando un messaggio sul topic to-notifier.

Questa separazione permette di scalare l'analisi dei dati indipendentemente dalla velocità di invio delle notifiche.

Notifier e Prometheus

- **Notifier:** È un puro *Consumer* Kafka dedicato all'invio di email SMTP tramite Gmail.
- **Prometheus:** È il quinto microservizio del cluster. Configurato tramite ConfigMap, esegue lo *scraping* periodico (ogni 10s) degli altri servizi per raccogliere metriche di corretto funzionamento del sistema, nonché delle risorse da lui utilizzate.

Shared Library

Per garantire la manutenibilità e scalabilità del sistema, tutto il codice comune è stato raggruppato in una libreria shared, importata da tutti i servizi Python. Questa libreria include:

- **handlers.py (Gestione Errori):** Un gestore centralizzato che cattura le eccezioni, uniformando il formato delle risposte di errore (JSON standard) e il logging.
- **config.py (Configurazione):** Classe che carica e valida le variabili d'ambiente (URL database, credenziali, topic Kafka, porte Flask), fornendo valori di default sicuri.
- **database.py (Persistence):** Gestisce le connessioni verso MongoDB e PostgreSQL, evitando il dispendio risorse di connessioni multiple.
- **grpc_utils.py:** Contiene i file generati da Protobuf (user_pb2) e le funzioni helper per avviare server e client gRPC.
- **circuit_breaker.py:** L'implementazione della logica a stati (Closed/Open/Half-Open) descritta sopra.
- **cache.py:** La logica di caching per l'At-Most-Once.
- **kafka_utils.py:** Una funzione di utilità che impedisce ai servizi di avviarsi se il broker Kafka non è ancora raggiungibile, prevenendo crash all'avvio del pod.
- **opensky.py:** Gestisce l'autenticazione e il rinnovo dei token verso le API esterne.
- **metrics.py:** Definisce in modo univoco le metriche Prometheus (Counter e Gauge) esposte dai vari servizi, evitando duplicazioni o conflitti nei nomi.

Continuità Funzionale e Riferimenti alla Documentazione Pregressa

È fondamentale sottolineare che la transizione da un ambiente Docker Compose a un'orchestrazione su Kubernetes ha riguardato esclusivamente l'infrastruttura di deploy e networking, mantenendo inalterata la logica applicativa interna dei microservizi sviluppata nelle iterazioni precedenti (1 e 2).

Pertanto, il comportamento funzionale del sistema rimane consistente con quanto già validato. Di conseguenza, in questo documento abbiamo scelto di non duplicare le evidenze grafiche relative a meccanismi già ampiamente documentati. Ci riferiamo nello specifico agli screenshot delle risposte JSON delle API REST e ai log di dettaglio dei singoli componenti, quali:

- Il tracciamento delle transizioni di stato del **Circuit Breaker** (Closed/Open/Half-Open) nel Data Collector;
- La logica di elaborazione e matching delle soglie eseguita dall'**Alert System**;
- La simulazione di invio notifiche con la stampa a video del payload email (Sender/Subject/Body) da parte del **Notifier**.

Per la verifica puntuale di queste funzionalità e per la consultazione degli screenshot relativi ai test di integrazione, si rimanda alle relazioni tecniche degli **Homework 1 e 2**, dove tali aspetti sono stati trattati in modo esaustivo. La presente relazione concentra le evidenze sperimentali sul nuovo livello di **Osservabilità e Monitoraggio** (Capitolo 6), che costituisce il vero valore aggiunto architetturale di questa terza iterazione.

Deploy e Orchestrazione su Kubernetes

La terza iterazione del progetto ha segnato il passaggio fondamentale verso un'architettura **Cloud-Native**. Abbiamo migrato l'intero sistema da un'orchestrazione locale basata su *Docker Compose* a **Kubernetes**, lo standard industriale per la gestione dei container. Questa transizione ci ha permesso di affrontare e risolvere problematiche tipiche degli ambienti di produzione, quali lo scheduling intelligente dei carichi, la gestione dichiarativa delle configurazioni, l'isolamento di rete e la persistenza dei dati in un ambiente volatile.

Per lo sviluppo e il testing locale è stato utilizzato **Kind** (*Kubernetes in Docker*), uno strumento che permette di eseguire nodi Kubernetes all'interno di container Docker, offrendo un'emulazione fedele di un cluster reale.

Configurazione della Topologia del Cluster

Invece di utilizzare la configurazione standard a nodo singolo, abbiamo definito una topologia personalizzata multi-nodo tramite il file `kind-config.yaml`. Questo ci ha permesso di simulare la separazione fisica delle responsabilità tipica dei data center:

1. **Nodo Control-Plane (Master):** Questo nodo ospita i componenti vitali di Kubernetes. Tramite la direttiva `kubeadmConfigPatches`, abbiamo configurato il *kubelet* di questo nodo con l'etichetta `ingress-ready=true` e mappato le porte 80 (HTTP) e 443 (HTTPS) dell'host sulle porte del container. Questo permette all'Ingress Controller di intercettare direttamente il traffico in ingresso, fungendo da gateway per l'intero cluster.
2. **Nodo Monitoring (Tier Dedicato):** Un Worker Node riservato esclusivamente a **Prometheus**. Tramite il meccanismo di `nodeSelector` e `labels` (`tier: monitoring`), abbiamo vincolato il server di monitoraggio su questo nodo. Questo garantisce che la raccolta delle metriche non degradi le prestazioni delle applicazioni business, e viceversa.
3. **Nodi Worker Applicativi (x2):** Due nodi dedicati uno esclusivamente al Data Collector, l'altro generico, dedicato all'esecuzione dei microservizi (User Manager, ecc.) e dei database, gestiti dinamicamente dallo scheduler di Kubernetes.

Strategia di Networking: Ingress Controller

L'accesso ai servizi dall'esterno è stato centralizzato abbandonando l'uso di *NodePort* (che espone porte casuali e difficilmente gestibili) in favore di **NGINX Ingress Controller**. L'Ingress agisce come un proxy inverso, gestendo il routing basato sui path URL definiti nella risorsa `ingress.yaml`.

Le regole di instradamento configurate sono le seguenti:

- **Path /users:** Il traffico viene instradato al servizio `user-manager-service` (porta 5000), permettendo la registrazione e il login.
- **Path /flights, /statistics, /interests:** Questi path sono raggruppati e instradati verso il `data-collector-service` (porta 5000), che funge da entry-point per tutte le operazioni relative ai dati aerei.
- **Path / (Root):** È stato configurato per instradare il traffico verso il servizio di monitoraggio `prometheus-service` (porta 9090). Questa scelta permette di accedere alla dashboard di Prometheus semplicemente digitando `localhost` nel browser.

Gestione Dichiarativa dei Workload (Deployments)

La definizione dei pod è gestita tramite il file `deployments.yaml`, che contiene le specifiche per i microservizi *stateless* (User Manager, Data Collector, Alert System, Notifier). Per ogni Deployment, abbiamo definito dei fix specifici per Python, come l'aggiornamento immediato dei log e anche per Prometheus.

Quality of Service (QoS) e Resource Limits

Per evitare il fenomeno del "*Noisy Neighbor*" (dove un container fuori controllo consuma tutte le risorse del nodo), abbiamo applicato i **Resource Quotas** a tutti i Deployment (inclusi Kafka e Prometheus). Ogni Pod ha una definizione precisa di:

- **Requests:** Quantità minima di CPU/RAM garantita (necessaria per lo scheduling).
- **Limits:** Tetto massimo di risorse utilizzabili. Se un microservizio (es. Kafka) tenta di superare il limite di memoria, Kubernetes interviene terminando il Pod per proteggere la stabilità dell'intero nodo.

Focus: Kafka in modalità KRaft

Seguendo le ultime best practice di Apache Kafka, abbiamo deployato il broker in modalità **KRaft** (Kafka Raft Metadata mode). Questa architettura semplifica il deployment e riduce il consumo di risorse del cluster, aspetto critico in un ambiente locale simulato. Kafka è gestito come un Deployment standard, trattando i messaggi in coda come transitori.

Configurazione Esterna

Per garantire la portabilità e la sicurezza, abbiamo disaccoppiato rigorosamente la configurazione dal codice sorgente.

ConfigMaps

Il file `configmap.yaml` centralizza tutte le variabili non sensibili di configurazione come nomi dei database, porte, ecc. Il tutto disaccoppiando il codice dell'applicazione dal deployment. In modo da poter cambiare facilmente le impostazioni senza dover aggiornare il codice. In poche parole, serve a configurare l'ambiente a runtime.

Secrets

Le informazioni critiche sono gestite tramite oggetti Secret (`secrets.yaml`), con i valori codificati in Base64:

- **Database:** Password di root per PostgreSQL (`postgres-secret`).
- **API Esterne:** Username e Password per l'autenticazione su OpenSky Network (`opensky-secret`).
- **Notifiche:** Credenziali SMTP per l'invio delle email (`email-secret`). Questi secret vengono montati come variabili d'ambiente solo all'avvio del container, minimizzando il rischio di esposizione.

Persistenza dello Stato

La natura effimera dei pod in Kubernetes comporta la perdita dei dati al riavvio del container. Per ovviare a questo, abbiamo configurato dei **PersistentVolumeClaim (PVC)** per i componenti *stateful*:

1. **PostgreSQL:** Un PVC garantisce che i dati degli utenti registrati sopravvivano al ciclo di vita del pod.
2. **MongoDB:** Un volume persistente è dedicato alla collezione dei voli e degli interessi utente.
3. **Prometheus:** Definito specificamente in `prometheus.yaml`, il PVC assicura che lo storico delle metriche raccolte non venga perso, permettendo analisi di trend anche a seguito di riavvii del sistema di monitoraggio.
4. **Redis:** E' gestito come StatefulSet per garantire la persistenza dei dati di lock e cache tra i riavvii, aumentando la robustezza del pattern At-Most-Once.

Sicurezza di Rete (Network Policies)

Per implementare un modello di sicurezza avanzato, abbiamo introdotto le **Network Policies** (file `network-policy-db.yaml`). In un cluster Kubernetes standard, tutti i pod possono comunicare liberamente tra loro. Noi abbiamo ristretto questo comportamento:

- **Isolamento PostgreSQL:** Una policy specifica consente il traffico in ingresso verso il DB *esclusivamente* dai pod con etichetta `app: user-manager`. Tentativi di connessione da altri pod vengono bloccati.
- **Isolamento MongoDB:** L'accesso è limitato ai soli servizi `data-collector` (scrittura) e `alert-system` (lettura).

Automazione del Deployment

Data la complessità dell'architettura (cluster custom, build immagini, loading su nodi virtuali), è stato sviluppato uno script di automazione `start.bat` che funge da pipeline di deployment. Lo script esegue le seguenti fasi critiche:

1. **Verifica Cluster:** Verifica l'esistenza del cluster kind. Se esiste già, salta la creazione per velocizzare il riavvio; altrimenti, ne crea uno nuovo applicando la configurazione dei nodi.
2. **Setup Ingress:** Installa il manifest ufficiale di NGINX, ma non prosegue immediatamente. Implementa una logica di attesa attiva (`kubectl wait ... --for=condition=ready`) con timeout di 90 secondi. L'Ingress Controller utilizza un *ValidatingWebhook* che impiega tempo per avviarsi. Applicare i manifest applicativi prima che l'Ingress sia pronto causerebbe errori di validazione e il fallimento del deploy.
3. **Build & Side-Loading delle Immagini:** Poiché Kind esegue i nodi come container Docker, questi non hanno accesso diretto alle immagini presenti sulla macchina host.
 - **Build:** Ricompila localmente le immagini con tag specifici (es. `data-collector:v3.5`).
 - **Kind Load:** Il comando `kind load docker-image` trasferisce fisicamente le immagini dall'host ai nodi del cluster. Questo passaggio è fondamentale per evitare che Kubernetes tenti (fallendo) di scaricare le nostre immagini private da internet.
4. **Deployment e Rollout:** Applica ricorsivamente tutti i manifest della cartella `k8s/` e forza un `kubectl rollout restart`. Questo garantisce che, anche se i deployment erano già attivi, i pod vengano ricreati per recepire l'ultima versione del codice appena caricata.

Monitoraggio e White-Box Monitoring

Per soddisfare i requisiti di **White-Box Monitoring** e garantire la piena osservabilità del cluster, abbiamo integrato **Prometheus** come componente infrastrutturale.

Abbiamo strumentato il codice Python utilizzando la libreria client ufficiale `prometheus_client`, definendo metriche personalizzate di tipo **Counter** (valori monotonicamente crescenti) e **Gauge** (valori che possono oscillare), arricchite con le label obbligatorie `service` e `node` per identificare la sorgente dei dati.

Metriche di Business

Queste metriche sono state definite specificamente per la logica del nostro dominio applicativo e ci permettono di capire "cosa sta facendo" il sistema in tempo reale.

Monitoraggio Data Collector

Il Data Collector è il componente che ingerisce i dati. Monitoriamo il ciclo di vita dello scheduler e il volume dei dati trattati.

Cicli di Fetch Totali (Counter)

- **Nome Metrica:** `data_collector_fetch_total`

- **Tipo:** Counter
- **Descrizione:** Conta il numero di volte che lo scheduler ha eseguito con successo una richiesta verso le API di OpenSky.
- **Analisi:** Il grafico mostra un andamento a "gradini" (step). Ogni gradino corrisponde a un ciclo di esecuzione dello scheduler (configurato ogni minuto). La regolarità dei gradini conferma che lo scheduler sta funzionando correttamente senza blocchi.



Figura 6.1: Andamento incrementale dei cicli di fetch eseguiti verso OpenSky.

Volume Dati Volo (Gauge)

- **Nome Metrica:** data_collector_last_flights
- **Tipo:** Gauge
- **Descrizione:** Indica il numero di voli (arrivi + partenze) recuperati e salvati su MongoDB nell'ultimo ciclo di esecuzione.
- **Analisi:** A differenza del counter, questo valore oscilla in base al traffico aereo reale. Nel grafico osserviamo un aumento netto (da circa 80 a oltre 120 voli), dimostrando la capacità del sistema di adattarsi a carichi di dati variabili.



Figura 6.2: Quantità di voli ingeriti per ogni ciclo di scraping.

Interessi Utente Attivi (Gauge)

- **Nome Metrica:** data_collector_total_interests
- **Tipo:** Gauge
- **Descrizione:** Traccia il numero totale di "Interessi" attualmente attivi nel sistema.
- **Analisi:** Questa metrica è fondamentale per dimensionare il sistema. Nel grafico vediamo un valore stabile, indicando che ci sono 3 configurazioni di monitoraggio attive processate ad ogni ciclo.



Figura 6.3: Numero totale di interessi attivi monitorati dal sistema.

Monitoraggio Notifier

Per il sottosistema di notifica, ci concentriamo sull'output effettivo verso gli utenti.

Email Inviato (Counter)

- **Nome Metrica:** notifier_email_sent_total
- **Tipo:** Counter
- **Descrizione:** Conta il numero totale di email di allerta consegnate con successo al server SMTP (Gmail).
- **Analisi:** Il grafico evidenzia i momenti esatti in cui sono scattati gli allarmi. La presenza della label type="alert_email" permette di distinguere il tipo di notifica inviata.



Figura 6.4: Conteggio cumulativo delle email di allerta inviate.

Monitoraggio User Manager

Per il gestore utenti, monitoriamo le operazioni CRUD critiche e la popolazione del database.

Registrazioni Utente (Counter)

- **Nome Metrica:** user_manager_registration_total
- **Tipo:** Counter
- **Label:** outcome (valori: created, duplicate_email).
- **Descrizione:** Traccia i tentativi di registrazione. L'uso delle label è cruciale: ci permette di distinguere i nuovi utenti (created) dai tentativi falliti o ripetuti (duplicate_email).

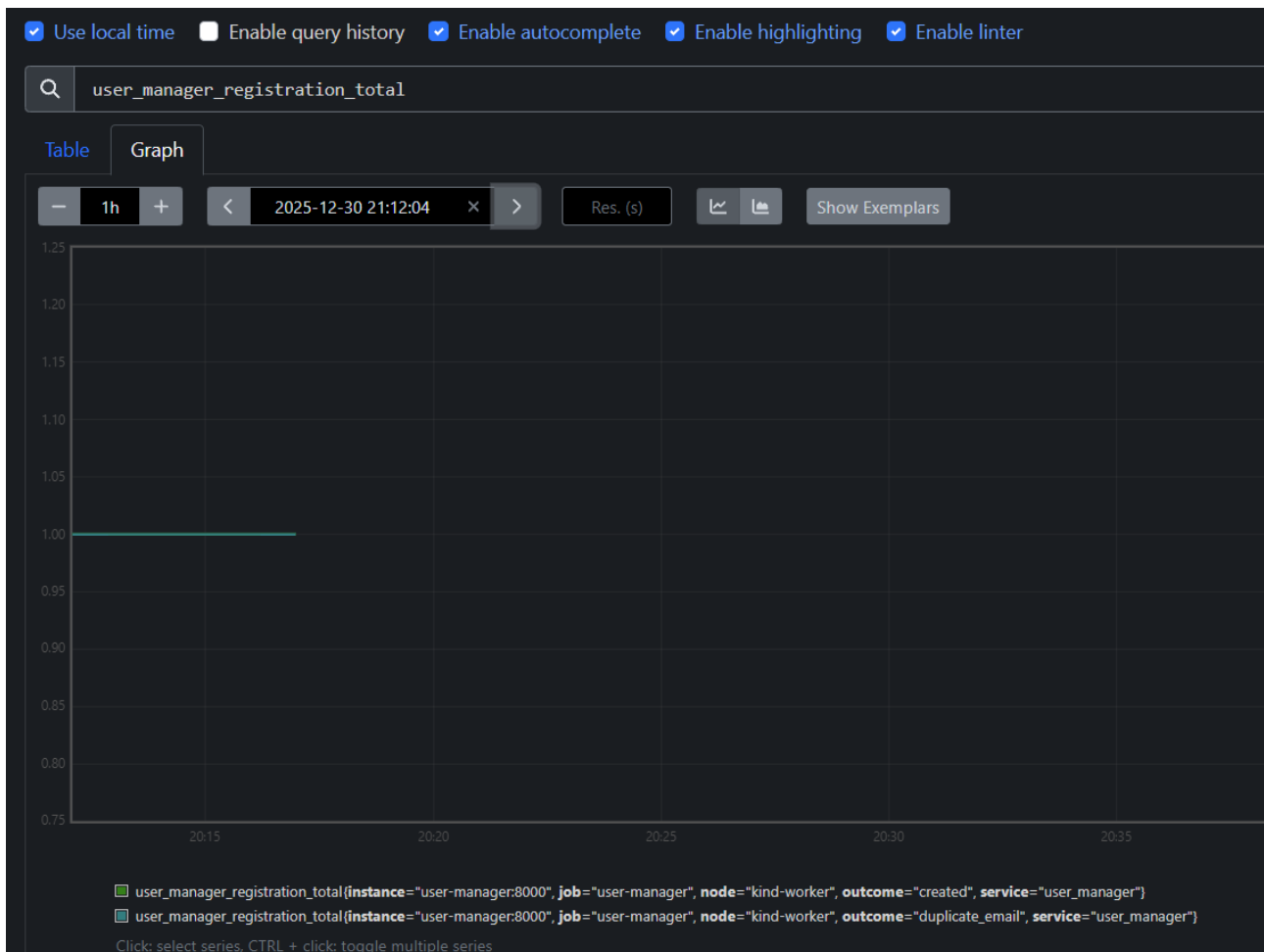


Figura 6.5: Monitoraggio delle registrazioni con distinzione dell'esito.

Cancellazione Utenti (Counter)

- **Nome Metrica:** user_manager_deletion_total
- **Tipo:** Counter
- **Label:** outcome="deleted".
- **Descrizione:** Conta il numero di utenti che hanno richiesto la cancellazione dell'account.
- **Analisi:** Nel grafico si nota chiaramente un "gradino" (step da 0 a 1), che corrisponde all'evento puntuale di una cancellazione andata a buon fine.

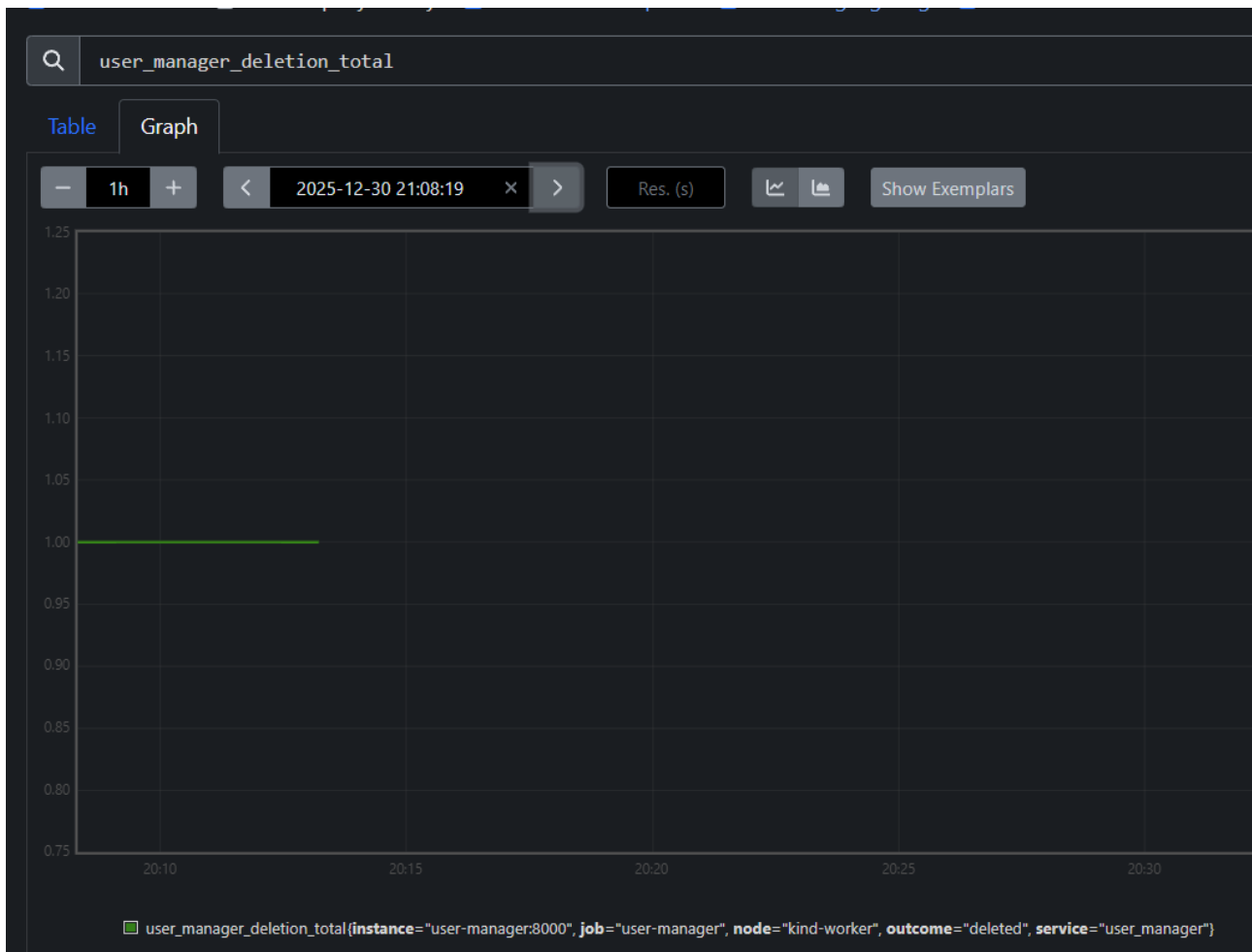


Figura 6.6: Evento di cancellazione utente rilevato dal sistema.

Totale Utenti Registrati (Gauge)

- **Nome Metrica:** user_manager_total_users
- **Tipo:** Gauge
- **Descrizione:** Indica il numero attuale di utenti presenti nel database PostgreSQL.
- **Analisi:** A differenza dei counter precedenti, questo valore può salire e scendere (registrazione vs cancellazione). Il grafico mostra una popolazione stabile di 2 utenti attivi nel periodo osservato.

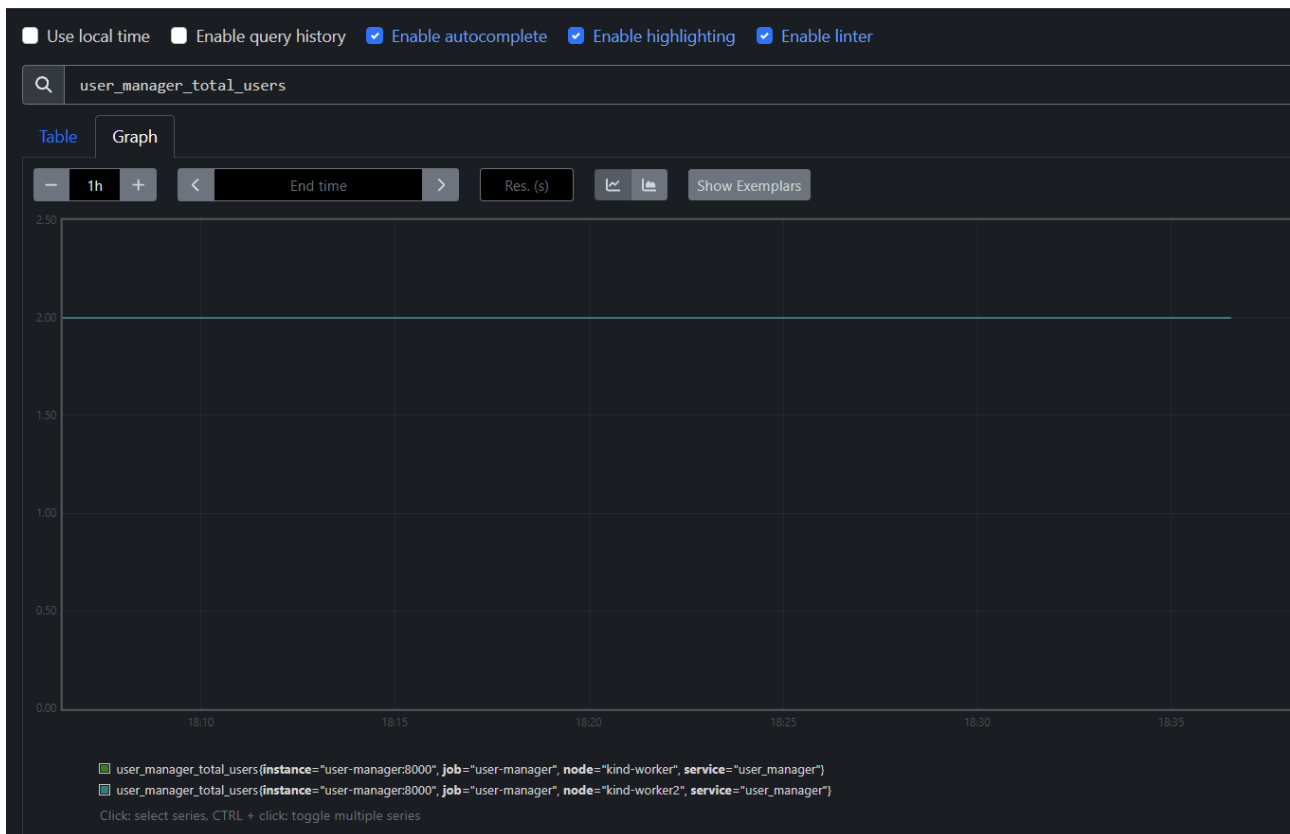


Figura 6.7: Numero totale di utenti attivi nel database.

Metriche di Sistema (Standard Metrics)

Oltre alle metriche di business, la libreria Prometheus espone automaticamente metriche standard per monitorare la salute dei processi Python e dei container.

Consumo Memoria RAM (Gauge)

- **Nome Metrica:** process_resident_memory_bytes
- **Analisi:** Dal grafico possiamo confrontare l'impronta di memoria dei diversi servizi. Si nota come il Data Collector (linea verde) e Prometheus (linea azzurra) abbiano un consumo maggiore rispetto a servizi più leggeri come il Notifier o lo User Manager.



Figura 6.8: Utilizzo della memoria RAM per i diversi microservizi.

Consumo CPU (Counter)

- **Nome Metrica:** process_cpu_seconds_total
- **Analisi:** I "salti" verso il basso (reset a zero) visibili nel grafico sono indicatori importanti: corrispondono ai momenti in cui i pod sono stati riavviati (ad esempio tramite il comando rollout restart del nostro script di avvio), azzerando i contatori interni del processo.



Figura 6.9: Tempo di CPU utilizzato. I reset indicano il riavvio dei Pod.

Uptime dei Processi (Gauge)

- **Nome Metrica:** `process_start_time_seconds`
- **Analisi:** Questo grafico mostra il timestamp di avvio. I gradini verticali confermano che i processi sono stati riavviati in sequenza, validando il funzionamento della nostra pipeline di deployment automatizzata.



Figura 6.10: Timestamp di avvio dei processi.

Metriche di Salute dell'Infrastruttura

Infine, monitoriamo lo stato di raggiungibilità dei target tramite la metrica sintetica generata da Prometheus stesso.

Disponibilità dei Servizi (UP)

- **Nome Metrica:** up
- **Tipo:** Gauge (binario: 1=UP, 0=DOWN).
- **Descrizione:** Questa è la metrica più critica per l'SRE (Site Reliability Engineering). Prometheus interroga periodicamente tutti i target configurati.
- **Analisi:** La tabella mostra il valore **1** per tutti i servizi (data-collector, notifier, user-manager, prometheus). Questo conferma che l'intero cluster è sano, che il Service Discovery di Kubernetes funziona correttamente e che tutti gli endpoint /metrics sono accessibili.

Evaluation time	up{instance="data-collector:8000", job="data-collector"}	up{instance="localhost:9090", job="prometheus"}	up{instance="notifier:8000", job="notifier"}	up{instance="user-manager:8000", job="user-manager"}
	1	1	1	1

Figura 6.11: Tabella di stato "UP" che conferma la salute di tutti i microservizi del cluster.

Performance dello Scraping

- **Metriche:** scrape_duration_seconds e scrape_samples_scraped.

- **Analisi:** I grafici confermano che Prometheus sta raccogliendo centinaia di campioni (samples) ad ogni ciclo e che la durata dello scraping è minima, indicando una bassa latenza di rete interna al cluster Kind.

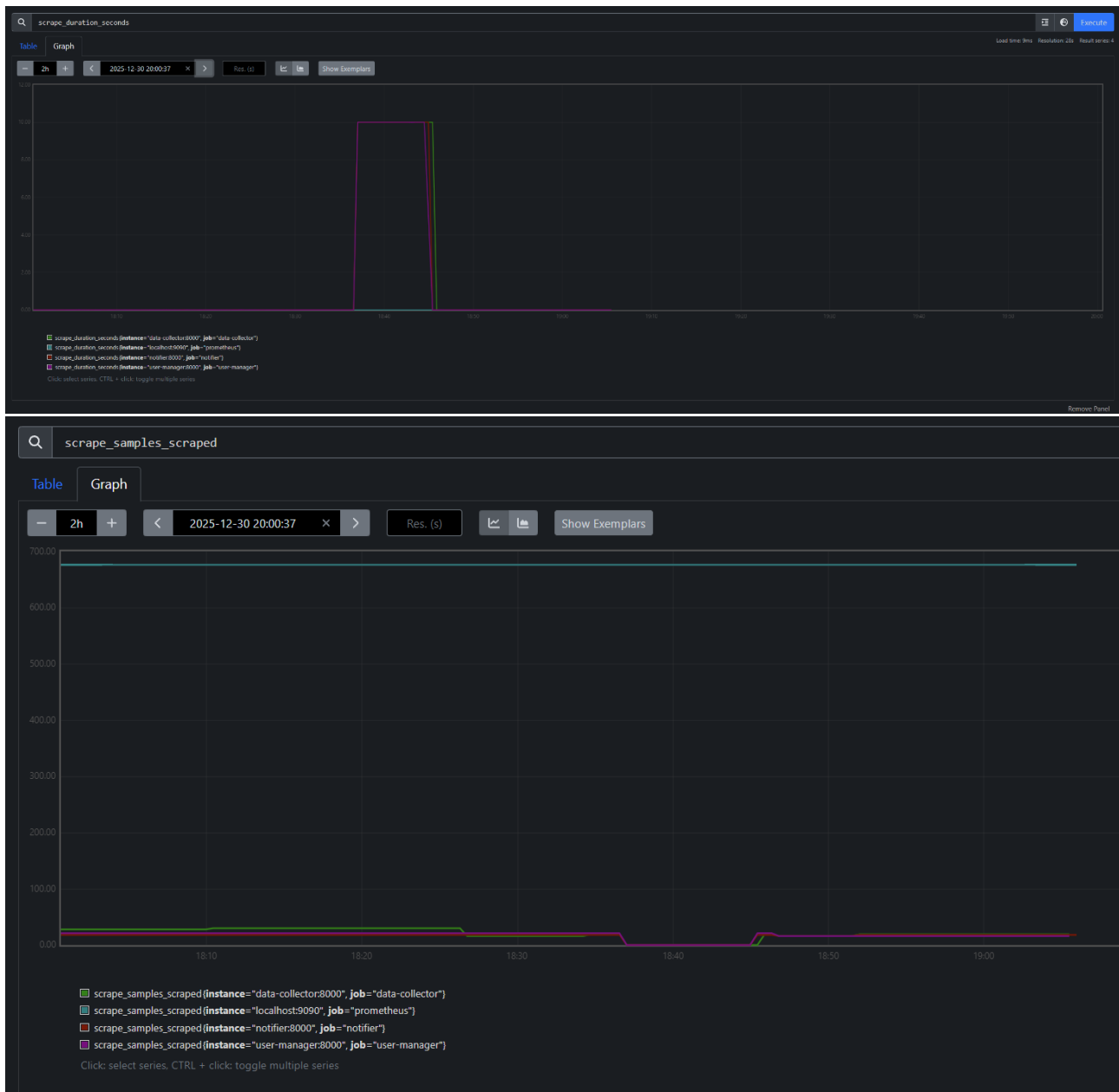


Figura 6.12 e 6.13: Dettaglio delle performance di scraping.

Conclusioni e Sviluppi Futuri

Il progetto OpenSky Flight Monitor rappresenta il punto di arrivo di un percorso evolutivo che ha trasformato una semplice applicazione client-server in un sistema distribuito Cloud-Native, resiliente e pienamente osservabile.

La terza iterazione (Homework 3) è stata determinante per comprendere le sfide della gestione di microservizi in produzione, spostando il focus dalla semplice implementazione della business logic (Python) alla gestione dell'infrastruttura che la ospita (Kubernetes).

Analisi dei Risultati Raggiunti

Il passaggio dall'orchestrazione locale tramite *Docker Compose* a un cluster **Kubernetes** (simulato tramite *Kind*) ha permesso di raggiungere obiettivi architetturali complessi:

Orchestrazione Dichiarativa e Self-Healing

A differenza dell'approccio imperativo, abbiamo definito lo "stato desiderato" del sistema tramite manifest YAML. Questo ha delegato a Kubernetes la responsabilità di mantenere il sistema attivo. Se il pod del data-collector dovesse crashare per un errore di memoria, il Controller Manager di Kubernetes lo riavvierebbe istantaneamente, garantendo la continuità del servizio senza intervento umano. L'adozione dell'**Ingress Controller NGINX** ha eliminato la gestione manuale delle porte (NodePort), fornendo un unico punto di accesso scalabile e sicuro.

Evoluzione dell'Event-Driven Design

Abbiamo consolidato l'uso di Apache Kafka non solo come buffer di messaggi, ma come spina dorsale dell'architettura. L'adozione della modalità KRaft (senza Zookeeper) ha semplificato la topologia del cluster, riducendo il consumo di risorse e allineando il progetto agli standard più moderni di streaming dati. La separazione netta tra *Producer* (Data Collector) e *Consumer* (Alert System/Notifier) ha dimostrato come disaccoppiare la velocità di ingestione (alta frequenza) dalla velocità di notifica (latenza variabile SMTP).

Osservabilità Completa (White-Box Monitoring)

L'integrazione di Prometheus ha trasformato il sistema da una "scatola nera" a una "scatola bianca". Non ci limitiamo a monitorare le risorse hardware (CPU/RAM), ma abbiamo visibilità sulla logica di business: sappiamo esattamente quanti voli vengono scaricati, quanti utenti si registrano e quante email partono. Le metriche *custom* (es. `user_registration_total` con label `outcome`) permettono di diagnosticare problemi applicativi (es. picchi di registrazioni fallite) che un monitoraggio infrastrutturale classico non avrebbe rilevato.

Limitazioni Attuali e Sviluppi Futuri

Nonostante il sistema soddisfi pienamente i requisiti, un'analisi critica evidenzia alcune aree di ottimizzazione necessarie per un ipotetico deploy in ambiente di produzione aziendale.

Alerting Proattivo

Al momento, Prometheus raccoglie le metriche e noi le visualizziamo passivamente tramite dashboard. Un amministratore deve guardare i grafici per accorgersi di un problema. Uno sviluppo futuro sarebbe quello di configurare **Prometheus Alertmanager** per definire regole di allerta. Questo permetterebbe al sistema di inviare notifiche al team di sviluppo non appena un microservizio degrada le prestazioni, chiudendo il cerchio dell'osservabilità.

Appendice A: Guida all'Installazione e Deploy

Al fine di semplificare il processo di build e deployment dell'intera infrastruttura, il progetto è corredato da una suite di script di automazione che astraggono la complessità dei comandi docker, kind e kubectl. L'infrastruttura sottostante è costituita da una topologia distribuita composta da **3 nodi**: un Control-Plane e due Worker Nodes, per simulare un reale ambiente di produzione ad alta disponibilità.

Di seguito vengono illustrate le procedure per l'avvio, l'arresto e il ripristino dell'ambiente per sistemi Windows e macOS/Linux.

Prerequisiti

Prima di eseguire gli script, assicurarsi che sulla macchina ospitante siano installati e correttamente configurati:

1. **Docker Desktop** (o Docker Engine su Linux/Mac).
2. **Kind** (Kubernetes in Docker) installato e aggiunto al PATH.
3. **Kubectl** (Kubernetes Command Line Tool).

Inizializzazione

Per il primo avvio, o nel caso in cui siano state apportate modifiche al codice sorgente Python che richiedono una ricompilazione delle immagini, utilizzare lo script di inizializzazione principale.

- **Windows:** Eseguire start.bat
- **macOS / Linux:** Eseguire sh start.sh (previa assegnazione permessi: chmod +x start.sh)

Funzionamento dello script: Lo script esegue una pipeline sequenziale completamente automatizzata:

1. **Cluster Provisioning:** Verifica l'esistenza del cluster kind. Se assente, crea i container Docker necessari per simulare la topologia a 3 nodi (kind-control-plane, kind-worker, kind-worker2) definita in k8s/kind-config.yaml.
2. **Ingress Setup:** Installa l'Ingress NGINX e attende attivamente (tramite kubectl wait) che il controller sia in stato *Ready* prima di procedere, prevenendo errori di validazione dei webhook.
3. **Build:** Eseguire il docker build delle 4 immagini dei microservizi con i tag di versione aggiornati.
4. **Load:** Carica le immagini locali direttamente all'interno dei nodi del cluster tramite kind load docker-image. Questo passaggio è critico poiché i nodi Kind non hanno accesso al registro immagini della macchina host.
5. **Apply & Rollout:** Applica i manifest Kubernetes e forza un riavvio dei pod (rollout restart) per garantire che venga eseguita l'ultima versione del codice.

Gestione Operativa (Stop & Resume)

Poiché l'avvio completo (build + deploy) può richiedere diversi minuti, sono stati predisposti script leggeri per "congelare" e riprendere il lavoro senza distruggere il cluster e senza perdere i dati persistenti.

Arresto del Cluster

Eseguendo questo script, il sistema invia il comando docker stop ai tre container che compongono il cluster (kind-control-plane, kind-worker, kind-worker2). In questo stato, il cluster non consuma risorse CPU/RAM, ma lo stato dei database (MongoDB, Postgres) e la configurazione di Kubernetes rimangono intatti su disco.

- **Windows:** Eseguire stop_cluster.bat
- **macOS / Linux:** Eseguire sh stop_cluster.sh

Ripristino del Cluster

Eseguendo questo script, i tre nodi vengono riavviati istantaneamente (docker start ...). Kubernetes rileva automaticamente il ritorno online dei nodi e ripristina lo scheduling dei pod esattamente nello stato in cui erano stati lasciati. Questa procedura permette di riprendere lo sviluppo in pochi secondi, evitando i tempi di attesa della fase di inizializzazione.

- **Windows:** Eseguire resume_cluster.bat
- **macOS / Linux:** Eseguire sh resume_cluster.sh

Guida Pratica alle Query Prometheus

Per facilitare le operazioni di monitoraggio e debugging del sistema attraverso la dashboard di Prometheus (accessibile su <http://localhost>), in questa sezione sono documentate le metriche esposte dai microservizi. Le metriche sono suddivise in due categorie: **Metriche di Business**, definite custom nel codice Python per monitorare la logica applicativa, e **Metriche di Sistema**, esposte automaticamente dalle librerie client per il controllo delle risorse.

Metriche di Business

Queste query permettono di analizzare il comportamento funzionale dei microservizi e i flussi di dati.

- **data_collector_fetch_total** (Type: *Counter*)
 - **Descrizione:** Conta il numero totale di cicli di fetch eseguiti dallo scheduler verso le API di OpenSky.
 - **Utilizzo:** Un andamento a "gradini" regolari indica che lo scheduler funziona correttamente. Una linea piatta indica un blocco del servizio.
- **data_collector_last_flights** (Type: *Gauge*)
 - **Descrizione:** Indica il numero esatto di voli (arrivi + partenze) recuperati e salvati su MongoDB durante l'ultimo ciclo di esecuzione.

- **Utilizzo:** Monitorare il volume di dati ingeriti. Valori pari a 0 potrebbero indicare problemi con le API esterne o aeroporti senza traffico.
- **data_collector_total_interests** (Type: *Gauge*)
 - **Descrizione:** Rappresenta il numero totale di configurazioni di monitoraggio attualmente attive nel sistema.
- **user_manager_registration_total** (Type: *Counter*)
 - **Descrizione:** Contatore incrementale delle richieste di registrazione ricevute.
 - **Utilizzo:** Grazie alla label outcome, è possibile distinguere i successi dai fallimenti.
- **user_manager_deletion_total** (Type: *Counter*)
 - **Descrizione:** Contatore delle richieste di cancellazione account andate a buon fine.
- **user_manager_total_users** (Type: *Gauge*)
 - **Descrizione:** Indica il numero puntuale di utenti registrati presenti nel database PostgreSQL.
- **notifier_email_sent_total** (Type: *Counter*)
 - **Descrizione:** Contatore delle email consegnate con successo al server SMTP.

Metriche di Sistema e Risorse

Queste query sono essenziali per il *White-Box Monitoring* delle risorse hardware e della salute dei processi.

- **process_resident_memory_bytes** (Memoria RAM)
 - **Descrizione:** Indica la memoria fisica (RSS) occupata dal processo Python.
 - **Interpretazione:** Se la linea sale costantemente senza mai stabilizzarsi o scendere, è indice di un *Memory Leak*.
- **process_cpu_seconds_total** (Utilizzo CPU)
 - **Descrizione:** È un contatore che accumula i secondi totali di utilizzo della CPU. Da solo non è significativo.
 - **Interpretazione:** Per capire il carico attuale ("quanto sta lavorando la CPU adesso"), è obbligatorio usare una funzione di derivata.
- **process_start_time_seconds** (Uptime)
 - **Descrizione:** Restituisce il timestamp (Unix Epoch) del momento di avvio del processo.
 - **Interpretazione:** Utile per rilevare riavvii imprevisti dei Pod.

Metriche di Salute Infrastrutturale

Metriche fondamentali per la *Site Reliability Engineering* (SRE).

- **up** (Liveness)
 - **Descrizione:** Metrica binaria. Valore 1 = Servizio UP; Valore 0 = Servizio DOWN/Irraggiungibile.
 - **Utilizzo:** È la metrica primaria per definire gli allarmi di disponibilità.
- **scrape_duration_seconds** (Latenza Scraping)
 - **Descrizione:** Tempo impiegato da Prometheus per scaricare i dati dalla pagina /metrics.
 - **Soglia:** Valori superiori a 1.0 secondi indicano che il microservizio è sovraccarico o bloccato.
- **scrape_samples_scraped** (Volume Metriche)
 - **Descrizione:** Numero di righe di dati esposte dal microservizio. Utile per verificare se un servizio sta esponendo una quantità eccessiva di metriche (Cardinality explosion).

Accesso ai Database

In un cluster Kubernetes, i servizi di backend come i database (postgres-service, mongo-service) sono esposti tramite servizi di tipo ClusterIP. Questo significa che possiedono un IP interno raggiungibile **solo** dagli altri pod all'interno del cluster, ma sono totalmente isolati rispetto alla macchina ospitante (Host).

Per poter ispezionare i dati, eseguire query di debug o visualizzare le tabelle direttamente dal proprio IDE (IntelliJ IDEA) durante lo sviluppo, è necessario utilizzare la tecnica del **Port Forwarding**.

Creazione del Tunnel (Port Forwarding)

Il comando `kubectl port-forward` crea un tunnel cifrato temporaneo che mappa una porta locale del computer (localhost) direttamente sulla porta del Pod nel cluster.

Per accedere ai database, è necessario aprire due terminali separati (o tab di PowerShell/Bash) e mantenere attivi i seguenti comandi:

Terminale 1: Tunnel per PostgreSQL

```
kubectl port-forward service/postgres-service 5432:5432
```

La porta locale 5432 viene instradata verso il servizio PostgreSQL.

Terminale 2: Tunnel per MongoDB

```
kubectl port-forward service/mongo-service 27017:27017
```

La porta locale 27017 viene instradata verso il servizio MongoDB.

Terminale 3: Tunnel per Redis

```
kubectl port-forward service/redis-service 6379:6379
```

La porta locale 6379 viene instradata verso il servizio Redis.

(Nota: Se i comandi restano "appesi" senza dare errori, significa che il tunnel è attivo. Non chiudere i terminali finché si desidera mantenere la connessione).

Configurazione Data Source su IntelliJ IDEA

Una volta stabiliti i tunnel, i database remoti del cluster rispondono come se fossero installati localmente su localhost. È quindi possibile configurarli nel tool *Database* di IntelliJ.

Configurazione PostgreSQL

1. Aprire il pannello **Database** a destra.
2. Cliccare su **Add Data Source, PostgreSQL**.
3. Inserire i seguenti parametri:
 - **Host:** localhost
 - **Port:** 5432
 - **User:** postgres
 - **Password:** (Inserire la password definita in secrets.yaml, es. postgrespassword)
 - **Database:** user_db
4. Cliccare su **Test Connection** per verificare.

Configurazione MongoDB

1. Cliccare su **Add Data Source, MongoDB**.
2. Inserire i seguenti parametri:
 - **Host:** localhost
 - **Port:** 27017
 - **Authentication:** Selezionare "User & Password".
 - **User:** admin
 - **Password:** (Inserire la password da secrets.yaml, es. adminpassword)
 - **Database (Auth):** admin
 - *Dettaglio Importante:* In MongoDB l'utente root è definito nel database di sistema admin. È obbligatorio specificare questo database nel campo di autenticazione per permettere il login.
3. Cliccare su **Test Connection**.

